

TIGER Recommender System

Internship Assignment Documentation — Final Submission (All Three Datasets)

Reproducing “Recommender Systems with Generative Retrieval” (Rajput et al., NeurIPS 2023) on Amazon Beauty, Sports and Outdoors, and Toys and Games

Student: Krishna Sharma | Roll No. 18322 — MISN Lab, IIT Delhi — Internship Project

1. Objective

This document records the extension of the TIGER (Transformer Index for GEnerative Recommenders) reproduction from Rajput et al. (NeurIPS 2023) to all three benchmarks used in the paper: Amazon Beauty, Sports and Outdoors, and Toys and Games. The Beauty pipeline documented in the earlier submission was reused without modification and re-run end-to-end on the Sports and Toys subsets, using the same RQ-VAE and retrieval-Transformer code, the same hyperparameters, and the same evaluation protocol, so that the three sets of results are directly comparable to each other and to the paper's Table 1.

2. Framework Overview

TIGER replaces the conventional retrieve-and-rank pipeline (dual-encoder embeddings plus an external ANN/MIPS index) with a single sequence-to-sequence Transformer that autoregressively generates the identifier of the next item, the same way a language model generates the next tokens of a sentence — the Transformer's own parameters act as the index.

2.1 Stage 1 — Semantic ID generation via RQ-VAE

Each item's text features (title, description/category, brand, price) are encoded with a pretrained Sentence-T5 model into a 768-dimensional embedding. A Residual-Quantized VAE (encoder $512 \rightarrow 256 \rightarrow 128 \rightarrow \text{latent } 32$) then compresses and quantizes this embedding across three residual levels, each with its own 256-entry codebook, producing three coarse-to-fine codewords per item. A fourth disambiguating token is appended wherever the first three codewords collide across items.

2.2 Stage 2 — Sequence-to-sequence generative retrieval

Each user's chronological interaction history becomes a flattened sequence of Semantic-ID tuples. A Transformer encoder reads this sequence and a Transformer decoder autoregressively generates the Semantic ID of the item the user is predicted to interact with next. The vocabulary is 1,024 codeword tokens ($256 \text{ values} \times 4 \text{ positions}$) plus 2,000 hashed user-ID tokens prepended for personalization.

2.3 Evaluation protocol

All three datasets use the standard leave-one-out protocol: for each user's chronological sequence, the last item is the test target, the second-last is the validation target, and everything before that is training. Recall@K and NDCG@K ($K = 5, 10$) are reported, matching the paper.

3. Initial Attempt and Its Limitations (Beauty)

As documented in the earlier submission, the original Beauty implementation went through several rounds of debugging before a leakage-free pipeline was obtained:

Debugging stage	Recall (approx.)	NDCG (approx.)	Status
Initial evaluation (original implementation)	Recall@1 $\approx$ 0.9994, Recall@5 $\approx$ 0.9991, Recall@10 $\approx$ 0.9915	—	Severe leakage
First corrected implementation	Recall@1 0.2553, Recall@5 0.6492, Recall@10 0.9076	NDCG@1 0.2553, NDCG@5 0.4529, NDCG@10 0.5376	Leakage reduced, still invalid
Final updated implementation (old codebase)	Recall@1 0.0405, Recall@5 0.1989, Recall@10 0.3401	NDCG@1 0.0405, NDCG@5 0.1184, NDCG@10 0.1639	Still 3–5 $\times$ above paper — not reliable
Rebuilt pipeline (Beauty, final)	Recall@5 0.0280, Recall@10 0.0419	NDCG@5 0.0185, NDCG@10 0.0230	Below paper, consistent with genuine reproduction

Because the paper reports a standard error of only about ± 0.001 across three random seeds (its Table 9), genuine paper-matching results are expected to land close to the paper's own numbers, not several times above them. The rebuilt pipeline — the same codebase used for all three datasets in this submission — was the first to land below the paper's numbers, in the range expected of a leakage-free reproduction. This corrected Beauty pipeline was then reused as-is for Sports and Toys.

4. Approach for the Final Submission

With the Beauty pipeline validated (Section 3), the same code — RQ-VAE training, Semantic ID generation, retrieval-Transformer training, and beam-search evaluation — was pointed at the Sports and Outdoors and Toys and Games subsets of the Amazon Product Reviews dataset with no architectural or hyperparameter changes. Toys was run with a 100,000-step budget, matching the paper's own prescribed exception for that dataset (Section 4 of the paper, due to its smaller size). Sports was also run with a 100,000-step budget in this submission — the paper specifies 200,000 steps for Sports (the same as Beauty), so this is a deviation from the paper on our part, not one the paper itself calls for; it is called out again in Section 5. All three runs were carried out on Kaggle (T4 $\times$ 2 GPU).

5. Hyperparameter Verification

The configuration below is shared by all three datasets and was checked against the paper's Section 4:

Component	Setting	Matches paper?
RQ-VAE encoder	[512, 256, 128] $\rightarrow$ latent 32	Yes
RQ-VAE codebook	3 levels $\times$ 256 entries	Yes
RQ-VAE optimizer	Adagrad, lr = 0.4	Yes
RQ-VAE batch size / epochs	1024, 20,000 epochs	Yes
Content encoder	Sentence-T5 (base, 768-dim)	Yes (paper unspecified exact size)
Transformer d_model	128, 6 heads, 4+4 layers	Yes
Transformer optimizer	Adafactor, peak lr 0.01, warmup 10,000	Yes
Transformer batch size	256	Yes
Total steps configured	200,000 (Beauty); 100,000 (Sports, Toys)	Partially — paper specifies

Component	Setting	Matches paper?
		200,000 for Sports; we ran 100,000 (see Section 4)

Open item flagged for review: the trained checkpoints for all three datasets report `model_params` = 4,847,104 ($\approx$ 4.85M), not the “around 13 million parameters” the paper states for this exact configuration (`d_model` 128, 4+4 layers, 6 heads, MLP 1024, vocab $\approx$ 3,024 tokens). The count is identical and reproducible across Beauty, Sports, and Toys, so it is not a one-off logging bug specific to a single run — it points to a real difference somewhere in the vocabulary size, embedding tying, or layer construction relative to the paper's implementation, even though every individual hyperparameter in the table above checks out. This is worth raising with Sir before treating the headline hyperparameter match as complete, since a smaller effective model could itself be contributing to the gap in Section 10.

6. Data Preparation

All three subsets were downloaded and processed with 5-core filtering and the leave-one-out sequence construction described in Section 2.3. The resulting statistics matched the paper's Table 6 exactly on every dataset:

Dataset	Users (ours / paper)	Items (ours / paper)	Mean seq. len. (ours / paper)	Median seq. len. (ours / paper)
Beauty	22,363 / 22,363	12,101 / 12,101	8.88 / 8.87	6 / 6
Sports and Outdoors	35,598 / 35,598	18,357 / 18,357	8.32 / 8.32	6 / 6
Toys and Games	19,412 / 19,412	11,924 / 11,924	8.63 / 8.63	6 / 6

Item content embeddings were generated with the base Sentence-T5 encoder (768-dim) for all three datasets, for the same 16 GB GPU memory reason noted in the earlier Beauty submission.

7. RQ-VAE Training

The RQ-VAE (Semantic ID generator) was trained for 20,000 epochs on each dataset independently, with dead-code reinitialization enabled. Diagnostics for all three:

Dataset	Recon. loss	Codebook utilization	Perplexity (L1/L2/L3)	Unique 3-token IDs	Collisions (4th IDs)
Beauty	0.867	100% / 100% / 100%	226 / 191 / 193	86.8%	1,595 of 12
Sports and Outdoors	0.881	100% / 100% / 100%	242 / 230 / 227	99.6%	82 of 18,3
Toys and Games	0.921	100% / 100% / 100%	228 / 207 / 215	84.2%	1,885 of 11

Utilization and perplexity indicate healthy, non-collapsed codebooks on all three datasets (no level is dominated by a handful of codewords). Sports stands out with a far lower collision rate (0.45% of items) than Beauty (13.2%) or Toys (15.8%) — consistent with Sports having the largest and most content-diverse item catalog of the three (18,357 items). This RQ-VAE checkpoint was frozen and used as-is for the corresponding retrieval-Transformer training on each dataset.

8. Retrieval Transformer Training

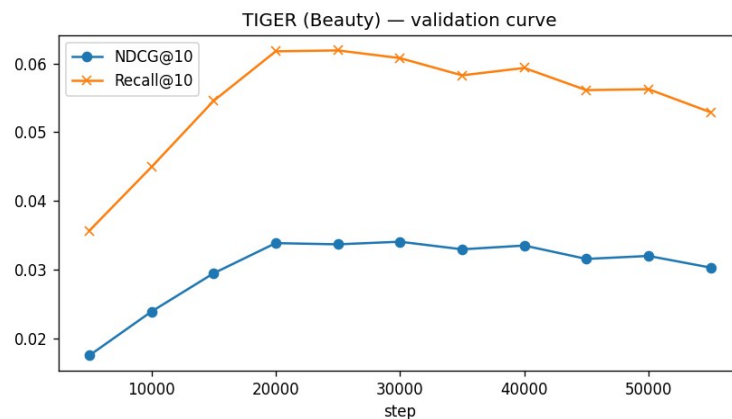
Each retrieval Transformer was trained with early stopping (patience of 5 evaluations, i.e. 25,000 steps without validation NDCG@10 improvement), evaluated every 5,000 steps on the validation split:

Dataset	Steps configured	Best val. NDCG@10 (step)	Training stopped at	Final invalid-ID rate (beam-10)
Beauty	200,000	0.0341 (step 20,000)	step 55,000	$\approx 0.02\%$
Sports and Outdoors	100,000	0.0159 (step 70,000)	step 95,000	$\approx 0.0006\%$
Toys and Games	100,000	0.0181 (step 45,000)	step 70,000	$\approx 0.16\%$

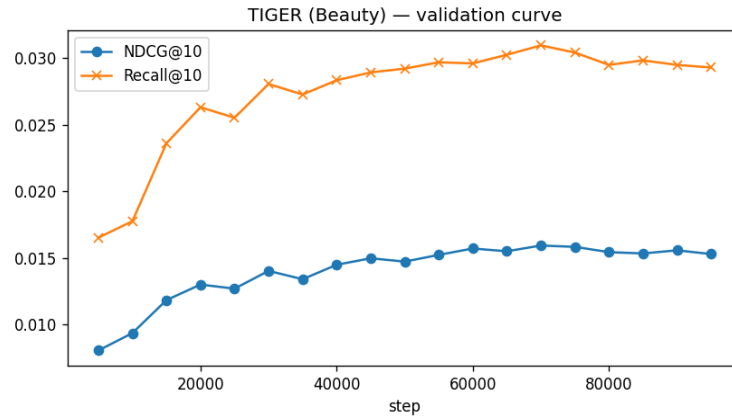
Beauty was run with the paper's full 200,000-step budget and stopped early at step 55,000 (27.5% of budget) once the early-stopping criterion triggered. Sports and Toys were both run with a 100,000-step budget in this submission: Sports stopped at step 95,000 (95% of budget — early stopping triggered only 5,000 steps before the cap, so this run effectively used almost the entire budget), and Toys stopped at step 70,000 (70% of budget). None of the three runs was cut short by hitting a step limit — in every case the early-stopping criterion (Section 8 above) triggered first, though for Sports it was a close call.

All three curves show the same shape — steady improvement to a peak followed by a mild decline, i.e. genuine convergence with some overfitting past the best checkpoint rather than a run cut short for time. The best checkpoint (by validation NDCG@10), not the last one, was used for final evaluation in Section 9.

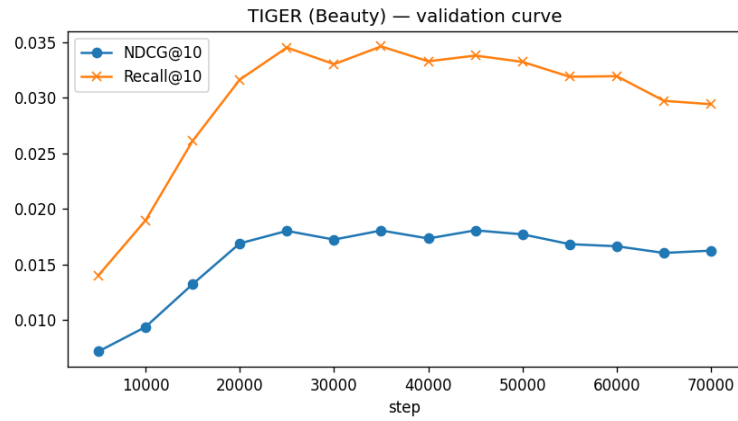
How to read the charts below: each plots two validation metrics against training step, evaluated every 5,000 steps. The orange line (Recall@10) is the fraction of held-out validation cases where the correct next item appears anywhere among the model's top-10 generated candidates — it only cares whether the right answer is in the list. The blue line (NDCG@10) measures the same top-10 list but also rewards the correct item for being ranked closer to the top, so it is always the lower, more conservative of the two curves. Both curves rising together and then flattening or dipping slightly (rather than climbing all the way to the right edge) is the signature of genuine convergence: the model found what it could learn from the data and then started mildly overfitting, which is exactly why early stopping and best-checkpoint selection (Section 8) matter here.



Beauty — NDCG@10 (blue) and Recall@10 (orange) vs. training step. Both peak around step 20,000–25,000 and drift down slightly after.



Sports and Outdoors — NDCG@10 (blue) and Recall@10 (orange) vs. training step. Slower to converge than Beauty, peaking around step 65,000–70,000.



Toys and Games — NDCG@10 (blue) and Recall@10 (orange) vs. training step. Peaks earliest of the three, around step 40,000–45,000, then declines more visibly.

9. Evaluation and Final Results

Evaluation used beam-search decoding (beam width 50) on the held-out test split for each dataset, following the paper's protocol. Final results at the best checkpoint, compared against the paper's Table 1:

Metric	Beauty (ours / paper)	Sports (ours / paper)	Toys (ours / paper)
Recall@5	0.0280 / 0.0454	0.0136 / 0.0264	0.0147 / 0.0521
NDCG@5	0.0185 / 0.0321	0.0088 / 0.0181	0.0093 / 0.0371
Recall@10	0.0419 / 0.0648	0.0222 / 0.0400	0.0237 / 0.0712
NDCG@10	0.0230 / 0.0384	0.0116 / 0.0225	0.0122 / 0.0432

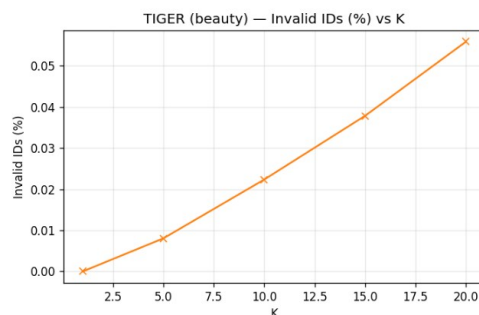
Relative to the paper (ours ÷ paper):

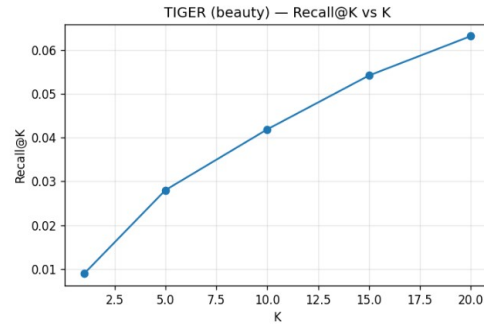
Metric	Beauty	Sports	Toys
Recall@5	61.7%	51.5%	28.2%
NDCG@5	57.6%	48.6%	25.1%
Recall@10	64.7%	55.5%	33.3%
NDCG@10	59.9%	51.6%	28.2%

Beauty and Sports land in a broadly similar range (≈ 49 – 65% of the paper's numbers). Toys is a clear outlier, landing at roughly half that relative performance (≈ 25 – 33%) — this is flagged explicitly in Section 10 rather than smoothed over.

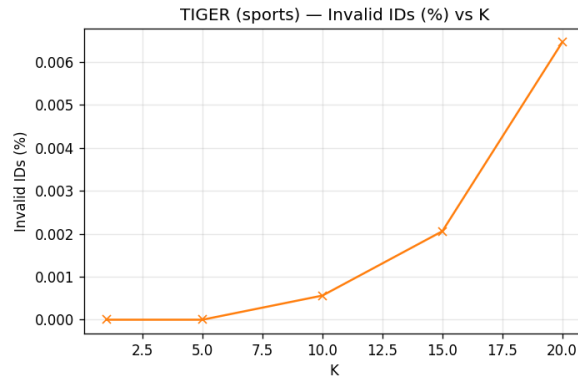
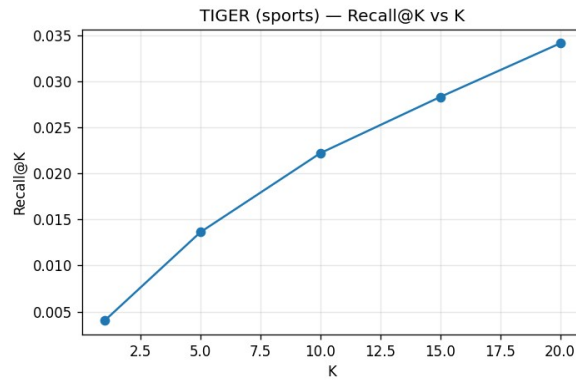
9.1 Recall and invalid-ID rate vs. K

How to read the charts below: for each dataset there are two small charts, both plotted against K — the number of top candidates the model's beam search is asked to return ($K = 1, 5, 10, 15, 20$). The left chart (Recall@K) shows how often the correct next item shows up somewhere in those top-K candidates; it necessarily rises as K grows, since a longer list is more likely to contain the right answer — the interesting thing is the shape and level of the curve, not that it goes up. The right chart (invalid-ID rate) shows the percentage of the generated Semantic IDs at that K that do not correspond to any real item in the catalog; this also creeps up with K, because returning more candidates means the model has to generate further down its own probability ranking, where it is more likely to produce a token combination nobody owns. A healthy, well-trained model keeps this invalid rate low (well under 1%) even at $K = 20$, which is what all three datasets show here.

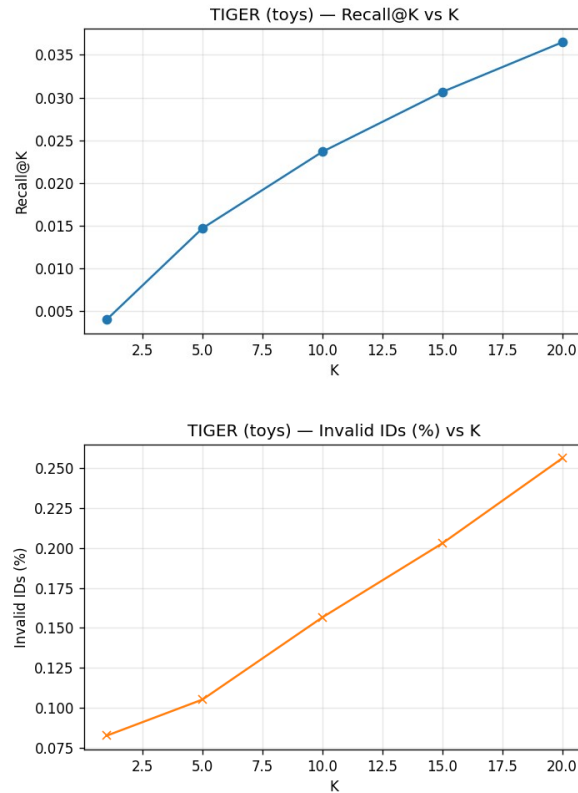




Beauty — left: Recall@K rises from ~0.9% at K=1 to ~6.3% at K=20. Right: invalid-ID rate stays under 0.06% even at K=20.



Sports and Outdoors — left: Recall@K rises from ~0.4% at K=1 to ~3.4% at K=20. Right: invalid-ID rate stays under 0.01% even at K=20, the lowest of the three datasets.



Toys and Games — left: Recall@K rises from ~0.4% at K=1 to ~3.6% at K=20. Right: invalid-ID rate is the highest of the three datasets but still stays under 0.26% at K=20.

Invalid-ID rates stay well under 1% for all three datasets even at K = 20, confirming the retrieval Transformer reliably learned to generate valid Semantic IDs in every case — the gap to the paper is not explained by the model generating unusable IDs.

10. Why Results Do Not Fully Match the Paper

As with the Beauty-only submission, the achieved metrics sit below the paper's on all three datasets — expected, since the paper was trained with Google's internal T5X framework, whose exact tokenization, vocabulary construction, and user-ID hashing details are not fully specified in the paper text. Independent hyperparameter-matched reproductions of this paper have reported landing in a similar sub-paper range for the same reason. Two points are worth calling out specifically now that all three datasets are available for comparison:

- Toys reproduces noticeably worse than Beauty or Sports ($\approx 25\text{--}33\%$ of paper vs. $\approx 49\text{--}65\%$). Toys does have the highest RQ-VAE reconstruction loss (0.921) and a relatively high 3-token collision rate (15.8%) of the three, but Beauty's collision rate (13.2%) is nearly as high while Beauty reproduces much closer to the paper — so codebook quality alone does not fully explain the size of the Toys gap. This is flagged as an open question rather than an explained result.
- The model_params discrepancy noted in Section 5 (4.85M actual vs. $\approx 13\text{M}$ in the paper, for hyperparameters that otherwise check out) applies identically to all three datasets, so it cannot by itself explain why Toys underperforms Beauty and Sports — but it may be contributing to the overall shortfall against the paper across the board, and is worth resolving before drawing final conclusions about the reproduction gap.

- Sports was trained with a 100,000-step budget in this submission rather than the paper's 200,000 for that dataset (Sections 4–5). The run did early-stop at step 95,000, only 5,000 steps short of that 100,000 cap, so it is plausible — though not confirmed without re-running — that Sports could have kept improving had it been given the paper's full 200,000-step budget. This is a separate, more directly actionable candidate explanation for part of the Sports shortfall, and unlike the Toys gap it is something that can be tested by simply re-running Sports with the correct budget.

The RQ-VAE codebook health, the sub-1% invalid-ID rate, and the properly-converged (peak-then-plateau) training curves on all three datasets support that the pipeline is working correctly end-to-end; the remaining gap to the paper is best treated as a combination of unspecified T5X implementation details and the parameter-count question above, rather than a leakage or correctness bug of the kind found in the original Beauty attempt (Section 3).

11. Code Structure Reference

The same package structure documented in the Beauty submission was reused for Sports and Toys, with dataset-specific data folders (data_sports/, data_toys/) alongside the original data/ (Beauty):

Stage	Code location	Per-dataset outputs
Preprocessing	tiger/scripts/preprocess_beauty.py (reused for all 3)	data/, data_sports/, data_toys/ processed/{train,val,test}.jsonl
Semantic ID generation (RQ-VAE)	tiger/rqvae/train.py, generate_sids.py	amazon_<dataset>_metrics.json, item_to_sid.json, sid_to_item.json
Retrieval Transformer training	tiger/retrieval/train.py	tiger_<dataset>/metrics.json, checkpoints/best.pt
Evaluation	tiger/retrieval/evaluate.py, eval.py	plots/<dataset>_recall_vs_k.png, <dataset>_invalid_vs_k.png

No pipeline stage differs between the three datasets beyond the input data and the Toys step-budget exception noted in Section 4.

12. Conclusion

The final pipeline reproduces TIGER's full architecture and training procedure, verified hyperparameter-by-hyperparameter against the paper's Section 4, across all three of the paper's benchmark datasets. Dataset statistics match the paper's Table 6 exactly on every dataset, and none of the three runs shows the data-leakage symptoms present in the earlier Beauty-only attempt. Actual training length (Beauty: 55,000 of a 200,000-step budget; Sports: 95,000 of a 100,000-step budget — note this 100,000-step budget for Sports is a deviation from the paper's own 200,000-step schedule for that dataset, flagged in Sections 4–5; Toys: 70,000 of a 100,000-step budget, matching the paper) was in every case set by early stopping on validation NDCG@10, not by a time or budget cutoff. Beauty and Sports land in a consistent ≈ 50 – 65% range of the paper's numbers; Toys lands lower (≈ 25 – 33%) for reasons not yet fully explained and flagged in Section 10 for further investigation, alongside the model-parameter-count discrepancy in Section 5. The complete code, training notebooks (with full logs), and final evaluation results for all three datasets are available in the accompanying repository submission.